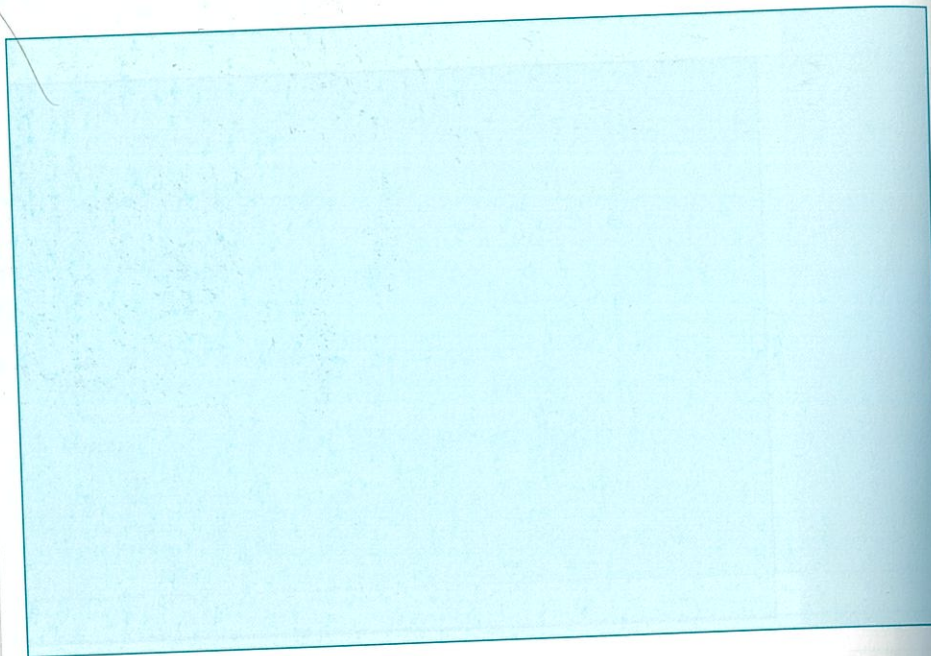


問 5

ユーザテーブル (usr)、アンケート回答テーブル (quest) それぞれに含まれるユーザ氏名を比較し、双方に含まれるデータのみを取り出してみましょう。



chapter

# 3

## データを登録／更新／削除 しよう

ここまでは、あらかじめ用意されたデータを検索する方法について紹介してきました。しかし、もちろん、データベースの役割は既存データを検索するばかりではありません。外部から新規のデータを挿入したり、変更されたデータを更新したり、はたまた、不要になったデータを削除したり。そのような日々発生するデータの変更を矛盾なくテーブルに反映させていくのも、データベースの役割です。

本章では、データの登録、更新、削除を行う方法について紹介していきます。



# 3-1 新規のデータを挿入したい

## INSERT 命令

MySQL PostgreSQL SQLite SQL Server Oracle



ここまでは、あらかじめ用意されたデータを検索する方法について紹介してきました。ここからは既存のテーブルに対してデータを挿入する方法についてみていくことにしましょう。



ユーザテーブル (usr) に新規ユーザの情報を入力してみましょう。追加する内容は以下の通りです。

### ●挿入するユーザ情報

|             |              |
|-------------|--------------|
| 利用者コード      | A200507      |
| 利用名 (氏)     | 鈴木           |
| 利用名 (名)     | 徳次郎          |
| 利用名 (氏カナ)   | スズキ          |
| 利用名 (名カナ)   | トクジロウ        |
| 住所 (都道府県名)  | 群馬県          |
| 住所 (市町村名)   | 群馬市北町        |
| 住所 (その他)    | 1-1-1        |
| 電話番号        | 040-999-9999 |
| E-Mail アドレス | NULL (未定義)   |



既存のテーブルに対して新規のデータを挿入するのは、INSERT 命令の役割です。

```
> INSERT INTO
>   usr
> VALUES
>   (
>     'A200507',
>     '鈴木',
>     '徳次郎',
>     'スズキ',
>     'トクジロウ',
>     '群馬県',
>     '群馬市北町',
>     '1-1-1',
>     '040-999-9999',
>     NULL
>   )
```

```
> ;
Query OK, 1 row affected (0.05 sec)
```

|         |         |
|---------|---------|
| user_id | A200507 |
| l_name  | 鈴木      |
| f_name  | 徳次郎     |
| ...     | ...     |
| email   | NULL    |

挿入

| ユーザテーブル (usr) |        |        |     |                     |
|---------------|--------|--------|-----|---------------------|
| user_id       | l_name | f_name | ... | email               |
| A200401       | 井上     | 一郎     | ... | inoue@wings.msn.to  |
| A200402       | 上原     | 幸一     | ... | uehara@wings.msn.to |
| A200403       | 江本     | 聡      | ... | emoto@wings.msn.to  |
| A200404       | 小野     | 雄二     | ... | ono@example.com     |
| A200405       | 河合     | 泰治     | ... | kawai@wings.msn.to  |
| A200507       | 鈴木     | 徳次郎    | ... | NULL                |



### 構文を理解しよう

INSERT 命令の基本的な構文は、以下の通りです。

### 構文 INSERT 命令 (1)

INSERT INTO テーブル名 VALUES (値,...)

挿入しなさい

どこへ?

なにを?

これを純粋に日本語に訳してみると、「<テーブル名>に対して<値,...>を挿入しなさい」というわけです。この場合、テーブル名は「usr」、値は列の定義順にカンマ区切りで指定しています。既に何度も述べていることですが、文字列/日付データはシングルクォート (') で括る必要があります。ただし、NULL 値はそれ自体が意味を持つキーワードですので、「NULL」ではなく「NULL」とする必要があります。「NULL」とした場合、本来の未定義という意味ではなく、「NULL」という文字列が挿入されてしまいますので、注意してください。



### SQL 命令を記述してみよう

INSERT 命令を記述する基本的な流れを辿ってみることにしましょう。

#### STEP 1 なにをしたいの?

SELECT 命令の場合と同様、まずはデータベースに対して「なにをしたいのか」を伝える必要があります。ここでは、データを挿入したいので、「INSERT INTO」を指定し



ます。

```
INSERT INTO
;
```

### STEP 2 どこに挿入したいの？

「挿入 (INSERT INTO) したい」といった場合、次に「どのテーブルに対してデータを挿入するのか」を示す必要があります。ここでは usr テーブルに対して挿入しますので、以下のように書いてみましょう。

```
INSERT INTO
usr
;
```

### STEP 3 挿入する値は？

最後に、usr テーブルに対してどの値を挿入するのかを、データベースに対して指定します。挿入する値は、VALUES 句の配下に値リストをカッコで括って指定します。値リストの指定順は、テーブル上で列が定義された順番です。

```
INSERT INTO
usr
VALUES
(
  'A200507',
  '鈴木',
  '徳次郎',
  'スズキ',
  'トクジロウ',
  '群馬県',
  '群馬市北町',
  '1-1-1',
  '040-999-9999',
  NULL
);
```

完成！

これで1つのSQL命令が完成です。

## マスタ・プラクティス



### 問 1

書籍情報テーブル (books) に対して、以下の内容で新規データを挿入してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

#### ●挿入対象のデータ

| 項目      | 値             |
|---------|---------------|
| ISBNコード | 4-8833-0000-4 |
| 書名      | SQL ミニ        |
| 価格      | 1000          |
| 出版社     | 日経BP          |
| 刊行日     | 2011-03-22    |
| 分類ID    | S2222         |

```
(
  ①
  books
  ②
  (
    '4-8833-0000-4',
    'SQL ミニ',
    1000,
    '日経BP',
    '2011-03-22',
    'S2222'
  )
);
```

### 問 2

著者情報テーブル (author) に対して、以下の内容で新規データを挿入してみましょう。

#### ●挿入対象のデータ

| 項目      | 値     |
|---------|-------|
| 著者ID    | I0001 |
| 著者名     | 井田健二  |
| 著者名(カナ) | イダケンジ |
| 生年月日    | NULL  |

```
INSERT INTO
  ①
VALUES
  ②
;
```



## 問 3

所属部署テーブル (depart) に対して、以下のデータを挿入するためのSQL命令を記述してみましょう。

## ●挿入対象のデータ

| 項目      | 値     |
|---------|-------|
| 所属部署コード | E03   |
| 所属部署名   | 第三営業部 |

## 問 4

以下は、商品テーブル (product) に対して、表のデータを挿入するためのSQL命令ですが、2点誤りがあります。誤りを指摘してください。

## ●挿入対象のデータ

| 項目    | 値          |
|-------|------------|
| 商品コード | SB00000001 |
| 商品名   | 黒スタンプ      |
| 単価    | 1250       |

```
INSERT
  product
VALUES
  (
    SB00000001,
    黒スタンプ,
    1250
  )
;
```

## 3-2

列指定で新規のデータを挿入したい  
INSERT 命令

MySQL PostgreSQL SQLite SQL Server Oracle



前項では、テーブルの全列に対して新規データを挿入する方法について紹介しました。この方法では、値が不要な（または決まっていない）列に対しては明示的に NULL 値を指定する必要があります。しかし、列数が多いテーブルでいちいち NULL 値を指定しなければならないのも冗長ですし、なにより間違いのもとです。



ユーザテーブル (usr) に新規ユーザの情報を入力してみましょう。追加する内容は、以下の通りです。

## ●挿入するユーザ情報

| 項目         | 値            |
|------------|--------------|
| 利用者コード     | B200506      |
| 利用者名 (氏)   | 神田           |
| 利用者名 (名)   | 愛            |
| 利用者名 (氏カナ) | カンダ          |
| 利用者名 (名カナ) | アイ           |
| 電話番号       | 040-888-8888 |



INSERT 命令では、特定列を指定した上でのデータ挿入が可能です。

```
> INSERT INTO
>   usr
>   (
>     user_id,
>     l_name,
>     f_name,
>     l_name_kana,
>     f_name_kana,
>     tel
>   )
> VALUES
>   (
>     'B200506',
>     '神田',
```



```

> '愛',
> 'カンダ',
> 'アイ',
> '040-888-8888'
> )
> ;
Query OK, 1 row affected (0.05 sec)

```

|         |              |
|---------|--------------|
| user_id | B200506      |
| l_name  | 神田           |
| f_name  | 愛            |
| ...     | ...          |
| tel     | 040-888-8888 |

挿入

| ユーザテーブル (usr) |        |        |        |              |          |
|---------------|--------|--------|--------|--------------|----------|
| user_id       | l_name | f_name | ...    | tel          | ...      |
| A200401       | 井上     | 一郎     | ...    | 040-111-0001 | ...      |
| A200402       | 上原     | 幸一     | ...    | 040-222-0002 | ...      |
| A200403       | 江本     | 聡      | ...    | 040-333-0003 | ...      |
| A200404       | 小野     | 雄二     | ...    | 040-111-0004 | ...      |
| A200405       | 河合     | 泰治     | ...    | 040-222-0005 | ...      |
| B200506       | 神田     | 愛      | (NULL) | 040-888-8888 | (デフォルト値) |

※未指定の列には NULL かデフォルト値をセット

## 構文を理解しよう

列を指定した INSERT 命令の基本的な構文は、以下の通りです。

### 構文 INSERT 命令 (2)

INSERT INTO テーブル名 (列名, ...) VALUES (値, ...)

挿入しなさい

どのテーブルに?

どの列に?

なにを?

例によってこれを日本語的に表してみると、「<テーブル名>の<列名, ...>に対して、対応する<値, ...>を挿入しなさい」となるわけです。本項の場合、テーブル名は usr で、列名は user\_id, l\_name, f\_name, l\_name\_kana, f\_name\_kana, tel だけに特定しています。指定されなかった列には、自動的に NULL、または列にあらかじめ定義されたデフォルト値 (P.239) がセットされます。もしも省略された列が NULL を許さず、デフォルト値も設定されていない場合には、INSERT 命令自体が失敗しますので、注意してください。

また、列名のセットと値のセットとは対応関係になければなりません。列セットと値セット

トの数が異なる場合には、INSERT 命令はやはり失敗します。たとえば、以下の SQL は列名のセット数と値のセット数が異なるため、失敗します。

```

> INSERT INTO
>   usr
>   (
>     user_id,
>     l_name,
>     f_name,
>     l_name_kana,
>     f_name_kana,
>     tel
>   )
> VALUES
>   (
>     'B200506',
>     '神田',
>     '愛',
>     '040-888-8888'
>   )
> ;

```

前項では、列名を省略するケースを紹介しましたが、列と値との対応関係を SQL 上で明示するという意味でも、列名は略記しないで記述することをお勧めします。

## NOTE

MySQL のような一部のデータベースでは、以下のように VALUES 句を列記することで、1 つの INSERT 命令で複数のレコードを同時に挿入することもできます。

```

INSERT INTO
  usr
  (
    user_id,
    l_name,
    f_name
  )
VALUES
  (
    'D200501',
    '山田',
    'リン'
  ),
  (
    'D200502',

```



```
'鈴木',
'朝子'
)
```

### SQL 命令を記述してみよう

列名を明示した INSERT 命令を記述する基本的な流れを辿ってみることにしましょう。

#### STEP 1 なにをしたいの？

まずは、データの挿入を意味する「INSERT INTO」命令を指定します。

```
INSERT INTO
;
```

#### STEP 2 どこに挿入したいの？

次に、データの挿入先であるテーブルとテーブル内の列を指定します。挿入先の指定は「テーブル名(列名,...)」の形式です。当然、列名のセットは指定したテーブルに存在する列でなければなりません。

```
INSERT INTO
usr
(
  user_id,
  l_name,
  f_name,
  l_name_kana,
  f_name_kana,
  tel
)
```

#### STEP 3 挿入する値は？

最後に、VALUES 句を使って、STEP 2 で指定した各列に対応する値をセットします。挿入する値のセットの数は、列名のセットの数と等しくなければなりません。

```
INSERT INTO
usr
(
  user_id,
  l_name,
  f_name,
  l_name_kana,
  f_name_kana,
  tel
)
VALUES
(
  'B200506',
  '神田',
  '愛',
  'カンダ',
  'アイ',
  '040-888-8888'
)
```

完成!

これで1つの SQL 命令が完成です。



## マスタ・プラクティス

## 問 1

アンケート回答テーブル (quest) に対して、以下の内容で新規データを挿入してみましょう。  
以下の空欄を埋めて、SQL命令を完成させてください。

## ●挿入対象のデータ

| 項目        | 値         |
|-----------|-----------|
| 回答者名      | 佐々木三郎     |
| 回答者名 (カナ) | ササキサブロウ   |
| 性別        | 男         |
| 都道府県      | 東京都       |
| 本書の評価     | 3         |
| 本書の感想     | 興味深い内容です。 |
| 回答日時      | (現在の日時)   |

```
INSERT INTO
quest
①
VALUES
②
;
```

## 問 2

貸し出し記録テーブル (rental) に対して、以下の内容で新規データを挿入してみましょう。  
以下の空欄を埋めて、SQL命令を完成させてください。

## ●挿入対象のデータ

| 項目       | 値             |
|----------|---------------|
| 利用者コード   | B200502       |
| ISBN コード | 4-0010-0000-0 |
| 貸出日      | (今日の日付)       |

```
①
rental
(
  user_id,
  isbn,
  rental_date
)
②
(
  'B200502',
  '4-0010-0000-0',
  CURRENT_DATE
)
;
```

## 問 3

著者情報テーブル (author) に対して、以下の内容で新規データを挿入してみましょう。

## ●挿入対象のデータ

| 項目       | 値          |
|----------|------------|
| 著者 ID    | U0001      |
| 著者名      | 内田幸子       |
| 著者名 (カナ) | ウチダサチコ     |
| 生年月日     | 1973-04-10 |

## 問 4

以下は、社員テーブル (employee) に対して、表のデータを挿入するためのSQL命令ですが、2点誤りがあります。誤りを指摘してください。

## ●挿入対象のデータ

| 項目      | 値       |
|---------|---------|
| 社員コード   | WA00001 |
| 社員名 (氏) | 和田      |
| 所属部署コード | J01     |
| 最終更新日   | (今日の日付) |

```
INSERT INTO
employee
(
  s_id
  l_name
  depart_id
  last_update
)
VALUE
(
  'WA00001',
  '和田',
  'J01',
  CURRENT_DATE
)
;
```



# 3-3

## 検索結果を他のテーブルに挿入したい INSERT 命令 / サブクエリ

MySQL PostgreSQL SQLite SQL Server Oracle



データを挿入するのは固定値によってばかりではありません。既存のテーブルから取り出したデータを、異なるテーブルに挿入することも可能です。



社員テーブル (employee) から最終更新日が 2012 年 1 月 1 日以降のものを取り出し、利用者テーブル (usr) に登録してみましょう。その際、フィールドの対応関係は以下の通りとします (email フィールドは固定値)。対応関係のないフィールドは NULL、またはデフォルト値がセットされます。

### ●フィールドの対応関係

|                      |         |
|----------------------|---------|
| employee             | usr     |
| s_id                 | user_id |
| l_name               | l_name  |
| f_name               | f_name  |
| 'admin@wings.msn.to' | email   |



INSERT 命令において、VALUES 句の代わりに SELECT 命令を使用します。

```
> INSERT INTO
>   usr
>   (
>     user_id,
>     l_name,
>     f_name,
>     email
>   )
>   SELECT
>     s_id,
>     l_name,
>     f_name,
>     'admin@wings.msn.to'
>   FROM
>     employee
>   WHERE
>     last_update >= '2012-01-01'
> ;
```

## 3-3 検索結果を他のテーブルに挿入したい

Query OK, 14 rows affected (0.03 sec)  
Records: 14 Duplicates: 0 Warnings: 0

| 社員テーブル (employee) |        |        |     |             |
|-------------------|--------|--------|-----|-------------|
| s_id              | l_name | f_name | ... | last_update |
| DA00001           | 大門     | 一郎     | ... | 2011-08-10  |
| FU00001           | 藤井     | 雄太     | ... | 2011-04-10  |
| AI00001           | 相沢     | 聡      | ... | 2012-02-24  |
| HA00002           | 葉山     | 俊輔     | ... | 2012-12-11  |
| HA00001           | 速水     | 和幸     | ... | 2007-02-03  |

指定列: s\_id, l\_name, f\_name  
条件句: last\_update >= '2012-01-01'  
条件に合致した行の指定列だけを抽出

| s_id    | l_name | f_name |
|---------|--------|--------|
| AI00001 | 相沢     | 聡      |
| HA00002 | 葉山     | 俊輔     |

email 列には固定値をセット

| s_id    | l_name | f_name | email              |
|---------|--------|--------|--------------------|
| AI00001 | 相沢     | 聡      | admin@wings.msn.to |
| HA00002 | 葉山     | 俊輔     | admin@wings.msn.to |

| ユーザテーブル (usr) |        |        |        |          |                     |
|---------------|--------|--------|--------|----------|---------------------|
| user_id       | l_name | f_name | ...    | ...      | email               |
| A200401       | 井上     | 一郎     | ...    | ...      | inoue@wings.msn.to  |
| A200402       | 上原     | 幸一     | ...    | ...      | uehara@wings.msn.to |
| A200403       | 江本     | 聡      | ...    | ...      | emoto@wings.msn.to  |
| AI00001       | 相沢     | 聡      | (NULL) | (デフォルト値) | admin@wings.msn.to  |
| HA00002       | 葉山     | 俊輔     | (NULL) | (デフォルト値) | admin@wings.msn.to  |

取得した結果を挿入

※未指定の列には NULL かデフォルト値をセット



### 構文を理解しよう

外部テーブルから取り出したデータを挿入する場合の INSERT 命令の構文は、以下の通りです。

#### 構文 INSERT 命令 (副問い合わせ)

INSERT INTO テーブル名 (列名, ...) SELECT 命令

挿入しなさい    どのテーブルに?    どの列に?    なにを?



つまり、「<テーブル名>の<列名 ...>に対して、< SELECT 命令>によって取り出した値を挿入しなさい」というわけです。この場合、テーブル名は usr とし、列名は user\_id, l\_name, f\_name, email です。指定された列名と、SELECT 命令によって取り出される列は、1:1 の関係になればなりません。また、固定値をセットしたい列に対しては、SELECT 命令の取得列で固定値を指定するだけです（ここでは email 列に対応する E-Mail アドレスがそれに該当します）。

このように、ある命令のもととなる問い合わせのことを、**サブクエリ**というのです。サブクエリは SELECT 命令に対してだけでなく、INSERT や UPDATE、DELETE 命令に対しても使用できます。

## NOTE

サブクエリを用いたデータの挿入は、あくまで既存のテーブルを対象としたものですが、それでは抽出結果を新規テーブルに挿入したい場合には、どのようにしたら良いのでしょうか。その場合、CREATE TABLE ...AS 命令を使用します。たとえば、書籍情報テーブル (books) から、出版社が「日経BP」の書籍のみを取り出し、その結果を nikkei テーブルに格納するには、以下のように記述します。

```
CREATE TABLE
  nikkei
AS
  SELECT
    *
  FROM
    books
  WHERE
    publish=' 日経BP '
```

データベース (SQL Server や Access など) によっては、SELECT INTO 命令を利用するものもあります。上記と同様の内容は、以下のように記述できます。

```
SELECT
  *
INTO
  nikkei
FROM
  books
WHERE
  publish=' 日経BP '
```

CREATE TABLE...AS 命令や SELECT INTO 命令を使用した場合、SELECT 命令の結果がそのまま新規テーブルの列定義となります。もしも新規テーブルの列に別名をつけたい場合には、AS 句で SELECT の取得列に別名をつけてください。



## SQL 命令を記述してみよう

サブクエリを利用した INSERT 命令を記述する基本的な流れを辿ってみることにしましょう。

### STEP 1 復習しよう

まずは、サブクエリとなる SELECT 命令を作成しておきます。SELECT 命令については、既に Chapter1、Chapter2 でも学習したとおりですので、ここでは復習も兼ねて自分の手で記述してみてください。

社員テーブル (employee) から最終更新日が 2012 年 1 月 1 日以降のものを取り出すための SQL 命令を作成します（取得列は s\_id, l\_name, f\_name、そして、固定値として 'admin@wings.msn.to' とします）。

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

この時点で、以下のような SQL 命令が記述できていれば正解です。これはまず横に置いておいて、続けて、以降のステップを見ていくことにしましょう。



```

SELECT
  s_id,
  l_name,
  f_name,
  'admin@wings.msn.to'
FROM
  employee
WHERE
  last_update >= '2012-01-01'
;

```

**STEP2** なにをしたいの？

まずは、データの挿入を意味する「INSERT INTO」命令を指定します。

```

INSERT INTO
;

```

**STEP3** どこに挿入したいの？

次に、データの挿入先であるテーブルとテーブル内の特定列を指定します。挿入先の指定は「テーブル名(列名,...)」の形式です。

```

INSERT INTO
  usr
(
  user_id,
  l_name,
  f_name,
  email
)
;

```

**STEP4** 挿入する値は？

最後に、挿入する値を取得するための SELECT 命令を追加します。SELECT 命令は、で作成したものを使用します。

```

INSERT INTO
  usr
(
  user_id,
  l_name,
  f_name,
  email
)
SELECT
  s_id,
  l_name,
  f_name,
  'admin@wings.msn.to'
FROM
  employee
WHERE
  last_update >= '2012-01-01'
;

```

完成!

これで1つのSQL命令が完成です。



## 問 1

社員テーブル (employee) から役職が主任、担当、アシスタントである社員のコードを取り出して、タイムカードテーブル (time\_card) に挿入してみましょう。その際、日付列には今日の日付を、勤務時間列には固定値で0をセットするものとします。

```
INSERT INTO
  time_card
(
  s_id,
  r_date,
  work_time
)
①
s_id,
CURRENT_DATE,
0
FROM
  employee
WHERE
  ②
;
```

## 問 2

店舗テーブル (shop) からすべての店舗コードを抜き出し、月間売り上げテーブル (sales) に対して挿入してみましょう。その際、売り上げ対象年月には今月の日付 (YYYY-MM) を、売上高には0を固定値でセットするものとします。

3-4 既存データを更新したい  
UPDATE 命令

MySQL PostgreSQL SQLite SQL Server Oracle



データの更新とは、テーブル上に既に存在するデータを適切な値に変更することを言います。商品テーブルであれば商品価格の変更、ユーザテーブルであれば住所や電話番号の変更、売り上げテーブルであれば売上高の修正等々、データを変更しなければならない局面は多く存在します。

書籍情報テーブル (books) の価格列は現在税抜きの価格となっています。そこで、登録されているすべての書籍に対して、消費税 5% を加味した価格に更新してみましょう。

既存のテーブルに対して特定の列を更新するのは、UPDATE 命令の役割です。

```
> UPDATE
>   books
> SET
>   price=price * 1.05
> ;
Query OK, 15 rows affected (0.05 sec)
Rows matched: 15  Changed: 15  Warnings: 0
```

| 書籍情報テーブル (books) |            |       |         |     |
|------------------|------------|-------|---------|-----|
| isbn             | title      | price | publish | ... |
| 4-0010-0000-0    | ハムスターの観察   | 900   | 山田出版    | ... |
| 4-7981-0959-2    | PEAR 入門    | 3000  | 翔泳社     | ... |
| 4-7980-0945-8    | PHP5 サンプル集 | 3000  | 秀和システム  | ... |
| 4-8833-0000-1    | SQL リファレンス | 2500  | 日経BP    | ... |
| 4-0010-0000-4    | フェレットの観察   | 1000  | 山田出版    | ... |

price 列に対して、一律 1.05 をかける

| 書籍情報テーブル (books) |            |       |         |     |
|------------------|------------|-------|---------|-----|
| isbn             | title      | price | publish | ... |
| 4-0010-0000-0    | ハムスターの観察   | 945   | 山田出版    | ... |
| 4-7981-0959-2    | PEAR 入門    | 3150  | 翔泳社     | ... |
| 4-7980-0945-8    | PHP5 サンプル集 | 3150  | 秀和システム  | ... |
| 4-8833-0000-1    | SQL リファレンス | 2625  | 日経BP    | ... |
| 4-0010-0000-4    | フェレットの観察   | 1050  | 山田出版    | ... |



## 構文を理解しよう

UPDATE 命令の基本的な構文は、以下の通りです。

### 構文 UPDATE 命令 (1)

UPDATE テーブル名 SET 列名 = 値, ...;

更新しなさい

どこの?

なにを?

例によって、これを純粋に日本語で表してみると、「<テーブル名>の<列名>を<値>で更新しなさい」というわけです。本項の場合、テーブル名には books を、列名は price で更新しなさい」というわけです。本項の場合、テーブル名には books を、列名は price のみを指定していますが、列名は複数指定することも可能です。列を複数指定する場合には、「列名 = 値」を必要な数だけカンマ区切りで記述します。

また、一律、同じ値に更新する場合には（もちろん）<値>の部分に固定値を指定してもかまいませんが、例題の「price \* 1.05」のように元の列の値を演算するような式として記述することも可能です。

## SQL 命令を記述してみよう

UPDATE 命令を記述する基本的な流れを辿ってみることにしましょう。

### STEP 1 なにをしたいの?

これまでの SELECT、INSERT 命令と同様、まずはデータベースに対して「なにをしたいのか」を伝える必要があります。ここでは、データを更新したいので、「UPDATE」を指定します。

```
UPDATE
;
```

### STEP 2 どのテーブルを更新したいの?

「更新 (UPDATE) したい」といった場合、次に「どのテーブルに対して更新したいのか」を示す必要があります。ここでは、books テーブルに対して更新するので、以下のように書いてみましょう。

```
UPDATE
  books
;
```

### STEP 3 更新対象の列は?

次に、更新する列を指定します。更新対象の列を特定するのは、SET 句の役割です。SET 句では、SET キーワードの後方に更新列を指定します。更新列が複数に及ぶ場合には、列名をカンマ (,) 区切りで記述してください。

```
UPDATE
  books
SET
  price
;
```

### STEP 4 更新後の値は?

最後に、更新列に対して更新後の新しい値を指定します。更新値は、固定値を指定しても、あるいは列のオリジナル値を基にした演算式を指定してもかまいません。

```
UPDATE
  books
SET
  price=price * 1.05
;
```

完成!

これで1つの SQL 命令が完成です。



## マスタ・プラクティス

問 1

書籍情報テーブル (books) を以下の要領で一括更新したいと思います。空欄を埋めて、正しいSQL命令を完成させてみましょう。

- ・ISBNコードの先頭に、固定値で「ISBN」を追加
- ・消費税込みの価格情報を税抜き価格に変更

```

①
books
SET
  ②
  ③
;

```

問 2

ユーザテーブル (usr) 上のメールアドレスをすべてNULL値でクリアしてみましょう。空欄を埋めて、正しいSQL命令を完成させてください。

```

UPDATE
  ①
SET
  ②
;

```

問 3

価格改訂に伴い、商品テーブル (product) に登録されている全商品を10%値上げすることになりました。対応するためのSQL命令を記述してみましょう。

```

--

```

問 4

問1でISBNコードの先頭に付加した固定文字列「ISBN」を一律取り除いてみましょう。対応するためのSQL命令を記述してください。

```

--

```

問 5

アンケート回答テーブル (quest) に「男性」「女性」「男」「女」と表記のゆらぎがありましたので、これを「男」「女」という表記に統一したいと思います。以下のSQL命令を記述しましたが、誤りがあり、正常に動作しません。誤りを2点、指摘してみましょう。

```

UPDATE INTO
  quest
VALUES
  sex=REPLACE(sex, '性', '')
;

```

問 6

社員テーブル (employee) において、役職列が部長である場合は「リーダ」に、課長である場合には「マネージャ」に名称変更してみましょう。

```

UPDATE
  employee
SET
  class=
  (
    ①
    class
    ② '部長' THEN 'リーダ'
    ② '課長' THEN 'マネージャ'
    ELSE ③
  )
END
;

```



## 3-5

# 特定の条件に合致するデータを更新したい UPDATE 命令

MySQL PostgreSQL SQLite SQL Server Oracle



前項では、テーブル内の全データに対して一律更新するパターンを紹介しましたが、一般的にはある特定のレコード（行）に対して更新を行うケースの方が多いでしょう。



アンケート回答テーブル (quest) から回答コードが「3」のデータについて、本書の評価を 3、本書の感想をブランクで更新してみましょう。また、その際に回答日時も現在日時に変更してください。



UPDATE 命令では、SELECT 命令同様、WHERE 句を指定することで更新対象の列を特定できます。

```
> UPDATE
>   quest
> SET
>   answer1=3,
>   answer2='',
>   answered=NOW()
> WHERE
>   id=3
> ;
Query OK, 1 row affected (0.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

| アンケート回答テーブル (quest) |     |         |            |                     |
|---------------------|-----|---------|------------|---------------------|
| id                  | ... | answer1 | answer2    | answered            |
| 1                   | ... | 3       | 重宝しています。   | 2012-10-10 10:11:12 |
| 3                   | ... | 1       | ちょっと難しいです。 | 2012-10-18 20:11:12 |
| 5                   | ... | 2       | わかりやすいです。  | 2012-10-29 21:18:17 |
| 8                   | ... | 3       | 買ってよかった。   | 2012-11-10 21:14:18 |
| 9                   | ... | 3       | NULL       | 2012-11-21 14:18:19 |

id 列が 3 の行の値を変更  
※id 列は主キーなので、更新行は 1 行のはず！

(次ページへ)

## 3-5 特定の条件に合致するデータを更新したい

(前ページから)

|          |        |
|----------|--------|
| answer1  | 3      |
| answer2  | "      |
| answered | (現在時刻) |

| アンケート回答テーブル (quest) |     |         |           |                     |
|---------------------|-----|---------|-----------|---------------------|
| id                  | ... | answer1 | answer2   | answered            |
| 1                   | ... | 3       | 重宝しています。  | 2012-10-10 10:11:12 |
| 3                   | ... | 3       | "         | (現在時刻)              |
| 5                   | ... | 2       | わかりやすいです。 | 2012-10-29 21:18:17 |
| 8                   | ... | 3       | 買ってよかった。  | 2012-11-10 21:14:18 |
| 9                   | ... | 3       | NULL      | 2012-11-21 14:18:19 |



## 構文を理解しよう

WHERE 句を指定した UPDATE 命令の基本的な構文は、以下の通りです。

### 構文 UPDATE 命令 (2)

UPDATE テーブル名 SET 列名= 値, ... WHERE 条件式;

更新しなさい    どの?    なにを?    どのような条件で?

つまり、「<テーブル名>で<条件式>に合致した行の<列名>列を<値>で更新しなさい」というわけです。この場合、テーブル名には「quest」を、列名は answer1、answer2、answered 列を、条件式には「id=3」を指定していますので、quest テーブルの回答コードが「3」のレコードについて、answer1、answer2、answered 列をそれぞれ「3」「(空文字)」「(現在の時刻)」で更新しなさい、ということになるわけです。

もちろん、WHERE 句には、論理演算子を含めた複合的な条件式を記述することも可能です。論理演算子については、今一度、P.71 を復習しておきましょう。



## SQL 命令を記述してみよう

条件付きの UPDATE 命令を記述する基本的な流れを辿ってみることにしましょう。

### STEP 1 復習しよう

既に UPDATE 命令でデータを更新する SQL 命令については学習済みです。ここでは、前項までの復習も兼ねて、ここまで紹介した書き順に沿って、まずは「アンケート回答テーブル (quest) の全データについて、本書の評価を 3、本書の感想をブランク、回答日時を現在の日時で更新して」みましょう。



|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

この時点で、以下のような SQL 命令が記述できていれば正解です。続けて、以降のステップを見ていくことにしましょう。

```
UPDATE
  quest
SET
  answer1=3,
  answer2='',
  answered=NOW()
;
```

### STEP 2 情報をどうしたいの？

デフォルトでは quest テーブルの全行を更新します。しかし、ここでは更新対象の行を特定したいので、絞り込みの条件を設定するための WHERE キーワードを指定します。

```
UPDATE
  quest
SET
  answer1=3,
  answer2='',
  answered=NOW()
WHERE
;
```

### STEP 3 どんな条件で？

WHERE キーワードは絞り込みを指定するだけの意味しか持たませんので、その直後に絞り込むための具体的な条件式を記述します。ここでは、「id 列が 3 である」ことを意味する条件式を記述します。

```
UPDATE
  quest
SET
  answer1=3,
  answer2='',
  answered=NOW()
WHERE
  id=3
;
```

完成！

これで 1 つの SQL 命令が完成です。



## マスタ・プラクティス

問 1

書籍情報テーブル (books) 上、出版社を「山田出版」から「WINGS出版」に変更してみましょう。空欄を埋めて、SQL命令を完成させてください。

UPDATE

 ①

SET

 ②

WHERE

 ③

;

問 2

社員テーブル (employee) 上、名前が「山田 奈美」さんの役職を「主任」に、最終更新日を今日の日付にしてみましょう。空欄を埋めて、SQL命令を完成させてください。

UPDATE

employee

SET

 ① , ②

WHERE

 ③

;

問 3

出版社「日経BP」が既存の書籍に対して、一律の価格改定（10%オフ）をしました。書籍情報テーブル (books) 上のデータも更新してみましょう。

問 4

貸し出し記録テーブル (rental) 上、貸出日が2012年3月31日より前で、かつ、現在貸し出し中であるレコードについて、returned列を9（紛失）で更新しましょう。

問 5

以下は、貸し出し記録テーブル (rental) 上、利用者コードがB200405、ISBNコードが4-0010-0000-1のデータについて、貸出日を今日の日付に更新するためのSQL命令ですが、誤りが2点あります。誤りを指摘してください。

UPDATE

rental

SET

rental\_date CURRENT\_DATE

WHERE

user\_id='B200405'

OR

isbn='4-0010-0000-1'

;



## 3-6

ある問い合わせ結果に基づいて、  
更新したい UPDATE 命令/サブクエリ

MySQL PostgreSQL SQLite SQL Server Oracle



INSERT 命令や SELECT 命令と同様に、UPDATE 命令でも他の問い合わせで得た結果に基づいて、更新値を決定することが可能です。

書籍分類テーブル (category) 上に登録されていない分類 ID を持つ書籍情報について、書籍情報テーブル (books) 上の分類 ID を「未分類 (Z9999)」に更新してみましょう。

サブクエリを利用することで、以下のように記述が可能です。

```
> UPDATE
> books
> SET
> category_id='Z9999'
> WHERE
> category_id
> NOT IN
> (
>     SELECT
>         category_id
>     FROM
>         category
> )
> ;
Query OK, 2 rows affected (0.03 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

## 3-6 ある問い合わせ結果に基づいて、更新したい

| 書籍情報テーブル (books) |          |       |     |             | 書籍分類テーブル (category) |               |
|------------------|----------|-------|-----|-------------|---------------------|---------------|
| isbn             | title    | price | ... | category_id | category_id         | category_name |
| 4-0010-0000-9    | SQL 入門   | 5500  | ... | S2222       | Z9999               | 未分類           |
| 4-0010-0000-1    | PHP ドリル  | 5100  | ... | P1111       | P1111               | PHP           |
| 4-0010-0000-5    | らくだの観察日記 | 1100  | ... | Z9999       | S2222               | SQL           |
| 4-0010-0000-6    | あひるの観察日記 | 700   | ... | A9999       | X3333               | XML           |
| 4-0010-0000-7    | かえるの観察日記 | 800   | ... | H9999       | J4444               | Java          |

抽出結果にないものについてのみ  
分類 ID を更新

| category_id |
|-------------|
| Z9999       |
| P1111       |
| S2222       |
| X3333       |
| J4444       |

全件データを抽出  
(サブクエリ)

| 書籍情報テーブル (books) |          |       |     |             |
|------------------|----------|-------|-----|-------------|
| isbn             | title    | price | ... | category_id |
| 4-0010-0000-9    | SQL 入門   | 5500  | ... | S2222       |
| 4-0010-0000-1    | PHP ドリル  | 5100  | ... | P1111       |
| 4-0010-0000-5    | らくだの観察日記 | 1100  | ... | Z9999       |
| 4-0010-0000-6    | あひるの観察日記 | 700   | ... | Z9999       |
| 4-0010-0000-7    | かえるの観察日記 | 800   | ... | Z9999       |



## 構文を理解しよう

サブクエリを用いた UPDATE 命令の基本的な構文は、以下の通りです。

## 構文 UPDATE 命令 (サブクエリ)

UPDATE テーブル名 SET 列名 = 値, ... WHERE 列名 演算子 (サブクエリ);

更新しない

どこの?

なにを?

どのような条件で?

P.165 のサブクエリと比べてみましょう。具体的な条件値の代わりにサブクエリを指定するところは、SELECT 命令の場合となんら変わるところはありません。サブクエリで抽出された値をもとに、条件式が動的に生成されるわけです。

本項の例であれば、「書籍分類テーブルから取り出した分類 ID のリスト」(これがサブクエリの部分です) に含まれない分類 ID を持つ書籍情報を洗い出し、その分類 ID を更新しています。



なお、今回の例は P.166 でも取り上げた EXISTS 演算子による **相関サブクエリ** としても記述が可能です。相関サブクエリで書き換えた SQL 命令は、以下の通りです。

```
UPDATE
  books
SET
  category_id='Z9999'
WHERE
  NOT EXISTS
  (
    SELECT
      *
    FROM
      category
    WHERE
      category.category_id=books.category_id
  )
;
```

なお、サブクエリは WHERE 句でのみ利用できるわけではありません。たとえば、SET 句に（相関）サブクエリを含めることで、外部テーブルの値をもとに更新できます。

### SQL 命令を記述してみよう

サブクエリを利用した UPDATE 命令を記述する流れは、以下の通りです。

#### STEP 1 復習しよう (1)

まずは、サブクエリとなる部分の SELECT 命令を記述してみましょう。サブクエリの部分はもっとも初歩的な SQL 命令ですので、ここまで学習を進めてきた読者の皆さんならば、すぐさまに記述できると思います。

それでは早速、「書籍分類テーブル category からすべての分類コードを取り出して」みましょう。

この時点で、次のような SQL 命令が記述できていれば正解です。これはまず横に置いておいて、続けて、以降のステップを見ていくことにしましょう。

```
SELECT
  category_id
FROM
  category
```

#### STEP 2 復習しよう (2)

続けて、サブクエリを受けて実行される本問い合わせを記述してみましょう。本問い合わせそのものも、これまでの復習にすぎませんので、これもこれまでの書き順にしたがって記述してみます。「分類 ID が x,y でない書籍情報の分類 ID を 'Z9999' に更新して」みましょう。x、y はこの時点で仮に置いておく条件値です。

この時点で、以下のような SQL 命令が記述できていれば正解です。

```
UPDATE
  books
SET
  category_id='Z9999'
WHERE
  category_id
NOT IN
  (
    x,y
  )
;
```



**STEP 3** 副問い合わせは？

ここまでできたら後はカンタン。**STEP 2** で仮置きしていた条件値「x,y」の部分に、

**STEP 1** で作成しておいたサブクエリを埋め込むだけです。

```
UPDATE
  books
SET
  category_id='Z9999'
WHERE
  category_id
NOT IN
(
  SELECT
    category_id
  FROM
    category
)
```

完成！

これで1つのSQL命令が完成です。

## マスタ・プラクティス



## 問 1

所属部署テーブル (depart) にはない所属部署に属する社員を社員テーブル (employee) から抽出し、所属部署コードを「Z99 (未登録)」に変更しましょう。  
以下の空欄を埋めて、SQL命令を完成させてください。

```
UPDATE
  employee
SET
  ①
WHERE
  depart_id
  ②
(
  SELECT
    ③
  FROM
    depart
)
```

## 問 2

問1のSQL命令を相関サブクエリを使って書き換えてみましょう。

```
①
employee
SET
  depart_id='Z99'
WHERE
  ②
(
  SELECT
    *
  FROM
    depart
  WHERE
    ③
)
```



問 3

書籍情報テーブル (books) 上、書名の後方に分類名を付加してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```
UPDATE
  books
SET
  ①
=
  CONCAT (
    ② ,
    (
      SELECT
        ③
      FROM
        category
      WHERE
        ④
    )
  )
;
```

問 4

貸し出し記録テーブル (rental) 上、returned列が9 (紛失) であるレコードについて、対応する書籍情報テーブル (books) 上の分類IDを「XXXXX」(その他)に更新してみましょう。

3-7

# 既存データを削除したい DELETE 命令

MySQL PostgreSQL SQLite SQL Server Oracle



ここまでデータの検索 (SELECT)、追加 (INSERT)、更新 (UPDATE) と学んできましたが、データ操作命令に含まれる最後の1つが削除 (DELETE) です。



月間売り上げテーブル (sales) から2012年11月以前のデータを削除してみましょう。



データを削除するのは、DELETE 命令の役割です。

```
> DELETE FROM
>   sales
> WHERE
>   s_date<='2012-11'
> ;
Query OK, 6 rows affected (0.05 sec)
```

| 月間売り上げテーブル (sales) |         |         |
|--------------------|---------|---------|
| s_id               | s_date  | s_value |
| M0001              | 2012-08 | 10784   |
| M0002              | 2012-10 | 12032   |
| H0001              | 2012-11 | 9975    |
| M0001              | 2012-12 | 11145   |
| H0001              | 2012-12 | 9861    |

2012年11月以前のデータ

| 月間売り上げテーブル (sales) |         |         |
|--------------------|---------|---------|
| s_id               | s_date  | s_value |
| M0001              | 2012-12 | 11145   |
| H0001              | 2012-12 | 9861    |

条件に合致したもののみを削除



## 構文を理解しよう

DELETE 命令の基本的な構文は、以下の通りです。

### 構文 DELETE 命令

DELETE FROM テーブル名 WHERE 条件式 ;

削除しなさい

どこを?

どのような条件で?



日本語で読み解くならば、「<テーブル名>から<条件式>に合致するデータを削除しなさい」となります。この場合、テーブル名は sales、条件式は「s\_date<='2012-11'」ですので、「sales テーブルから売り上げ月が 2012 年 11 月以前のデータを削除しなさい」という意味になるわけです。WHERE 句の条件式には、SELECT や UPDATE 命令と同様の構文で条件式を利用可能ですので、詳しくは P.56 を参照してください。

なお、WHERE 句を省略することも可能ですが、その場合、該当するテーブルの内容はすべて削除されます。たとえば、以下のように記述した場合には、books テーブルの内容がすべて削除されます。

```
DELETE FROM books;
```

こうした全件削除はもちろん、一般的に DELETE 命令を用いたデータを削除する場合は、十分に「そのデータが本当に不要なのか」「削除条件は妥当なのか」を検討する必要があります。多くのデータベースは、誤って削除したデータを復旧させるための手段を提供していますが、もちろん、誤ったデータを削除しないに越したことはありません。

もしも将来的に少しでもデータを利用する可能性が残っている（または削除して良い自信がない）のであれば、たとえばテーブル上に削除フラグなどの列を用意しておいて、不要となったデータであることを明示するなどの方法を探ることをお勧めします。

#### 削除フラグの利用も考えよう

| アンケート回答テーブル (quest) |      |     |     |                     |
|---------------------|------|-----|-----|---------------------|
| id                  | name | sex | ... | answered            |
| 1                   | 山田太郎 | 男   | ... | 2012-10-10 10:11:12 |
| 2                   | 井上茉莉 | 女   | ... | 2012-10-16 7:15:14  |
| 3                   | 上原遥  | 女   | ... | 2012-10-18 20:11:12 |
| 4                   | 江本修子 | 女   | ... | 2012-10-24 21:14:18 |
| 5                   | 小野博美 | 女   | ... | 2012-10-29 21:18:17 |

→いったん削除したデータは元に戻せない

| アンケート回答テーブル (quest) |      |     |     |                     |         |
|---------------------|------|-----|-----|---------------------|---------|
| id                  | name | sex | ... | answered            | deleted |
| 1                   | 山田太郎 | 男   | ... | 2012-10-10 10:11:12 |         |
| 2                   | 井上茉莉 | 女   | ... | 2012-10-16 07:15:14 |         |
| 3                   | 上原遥  | 女   | ... | 2012-10-18 20:11:12 | 1       |
| 4                   | 江本修子 | 女   | ... | 2012-10-24 21:14:18 |         |
| 5                   | 小野博美 | 女   | ... | 2012-10-29 21:18:17 |         |

→フラグだけなので、いつでもデータを復旧できる

また、行全体ではなく、特定のカラムのみをクリアしたいというケースでは、UPDATE 命令を使用します。DELETE 命令はあくまで行全体を削除する命令です。

## NOTE

テーブル内のデータを全件削除する方法として、WHERE 句をつけない DELETE 命令を使用する他に、TRUNCATE 命令を利用することも可能です。TRUNCATE 命令は「テーブルの切り捨て」を行うための命令で、無条件でテーブル内の全データを削除します。たとえば、以下は books テーブルの内容を全削除するための TRUNCATE 命令です。

```
> TRUNCATE TABLE books;
Query OK, 14 rows affected (0.13 sec)
```

WHERE 句をつけない DELETE 命令と何が違うのかというと、DELETE 命令が削除処理時に削除ログを記録するのに対して、TRUNCATE 命令はログを残しません。これによって、TRUNCATE 命令は削除処理をより高速に行うことができます。しかし、データベースは処理時のログを利用することで、もしなにかシステム障害などが原因で処理ミスが発生した場合にもデータを元に戻せるようにしています。ログが存在しないということは、万が一の場合に「復旧がきかない」ということを意味しますので、注意してください。

なお、SQLite では TRUNCATE 命令は利用できません。



## SQL 命令を記述してみよう

DELETE 命令を記述する基本的な流れを述べてみることにしましょう。

### STEP 1 なにをしたいの？

これまで同様、まずはデータベースに対して「なにをしたいのか」を伝える必要があります。ここでは、データを削除したいので、「DELETE FROM」を指定します。

```
DELETE FROM
;
```

### STEP 2 どのテーブルのデータを削除したいの？

「削除 (DELETE) したい」といった場合、次に「どのテーブルに対して削除したいのか」を示す必要があります。ここでは、sales テーブルに対して削除したいので、以下のように書いてみましょう。



```
DELETE FROM
sales
;
```

#### STEP 3 削除する条件は？

最後に、削除の対象となる行を WHERE 句で特定しておきましょう。条件式の書き方については、P.56 も合わせてご覧ください。

```
DELETE FROM
sales
WHERE
s_date<='2012-11'
;
```

完成！

これで1つのSQL命令が完成です。

#### マスタ・プラクティス



##### 問1

貸し出し記録テーブル (rental) から貸出日が2011年12月31日以前で、かつ、返却済みである情報について削除してみましょう。空欄を埋めて、正しいSQL命令を完成させてください。

```
DELETE FROM
rental
WHERE
②
;
```

##### 問2

アクセス記録テーブル (access\_log) からアクセス日時が2012年06月01日より前か、リンク元のURLが空である情報を削除しましょう。空欄を埋めて、正しいSQL命令を完成させてください。

```
DELETE FROM
①
WHERE
②
;
```

##### 問3

社員テーブル (employee) 上、退職済みで、最終更新日が2007-03-31以前のデータを削除してみましょう。

```
DELETE FROM

```



## 問 4

以下は、書籍情報テーブル (books) 上で分類IDが「Z9999 (未分類)」であるか、NULLである書籍を削除するSQL命令ですが、誤りが2点あります。誤りを指摘してください。

```
DELETE
  books
WHERE
  category_id='Z9999'
AND
  category_id IS NULL
;
```

## NOTE

データの挿入や更新、削除に関わる複数の SQL 命令を実行する場合、これを 1 つの処理として管理したい場合があります。

たとえば、銀行の振り込み処理を思い浮かべてみると分かりやすいかもしれません。銀行の振り込み処理を考えてみると、大きく以下の 2 つの処理から構成されます。

1. 自分の口座から指定金額を減額する
2. 振込先の口座に対して指定金額分を増額する

このような処理において、もしも 1. の処理が成功したのに、2. の処理は失敗したとしたらどうでしょう。自分の口座の残高は減ったのに、相手の口座の残高は増えないという哀しい結果になってしまいます。逆に、2. の処理が成功したのに、1. の処理が失敗したとしたら？振り込んだ人間にとっては嬉しい結果になりますが、システム的には致命的な不整合が発生することは言うまでもありません。

このように、SQL 命令としては複数あるが、意味的にはひとかたまりとして処理されなければならない処理、または、処理するためのしくみのことを**トランザクション**と言います。トランザクションを利用することで、更新は最終的にすべての作業が成功するまで確定(**コミット**)されません。つまり、もしも一連の作業の途中で処理に失敗した場合には、処理を取り消し、更新してしまったデータを元の状態に戻す(**ロールバック**する、といいます)ことができるというわけです。

データの挿入、更新、削除処理を行う場合には、トランザクション処理は必須であるといえます。

トランザクションを利用するには、MySQL の場合には次のようにします (ただし、

MySQL データベースでは、テーブルの型によってはトランザクションが利用できませんので、注意してください。一般的には InnoDB 型を採用してください。

```
> BEGIN;
>
> ... 複数の INSERT、UPDATE、DELETE 命令 ...
>
> COMMIT;      または ROLLBACK;
```

**BEGIN** 命令によって、トランザクションが開始されます。そして、これまで学んだ **INSERT**、**UPDATE**、**DELETE** 命令を発行した後、**COMMIT** 命令を実行すると変更が確定され、**ROLLBACK** 命令を実行すると変更が元に戻されます。たとえば、以下の例を見てみましょう。

```
> BEGIN;
Query OK, 0 rows affected (0.00 sec)

> INSERT INTO books(isbn, title) VALUES('4-0010-0000X',
'SQL 入門 2'); (注: 1 行で入力してください)
Query OK, 1 row affected (0.00 sec)
```

```
> SELECT isbn, title FROM books;
```

| isbn                     | title               |
|--------------------------|---------------------|
| 4-0010-0000-1            | PHP ドリル             |
| 4-0010-0000-9            | SQL 入門              |
| <del>4-0010-0000-X</del> | <del>SQL 入門 2</del> |
| 4-7980-0522-3            | JSP リファレンス          |
| 4-7980-0945-8            | PHP5 サンプル集          |
| 4-7981-0722-0            | XML 辞典              |
| 4-7981-0959-2            | PEAR 入門             |
| 4-8833-0000-1            | SQL リファレンス          |
| 4-8833-0000-2            | SQL プチブック           |
| 4-8833-0000-3            | XML リファレンス          |

10 rows in set (0.00 sec)

```
> ROLLBACK;
Query OK, 0 rows affected (0.05 sec)
```

```
> SELECT isbn, title FROM books;
```

| isbn | title |
|------|-------|
|------|-------|



|               |            |
|---------------|------------|
| 4-0010-0000-1 | PHP ドリル    |
| 4-0010-0000-9 | SQL 入門     |
| 4-7980-0522-3 | JSP リファレンス |
| 4-7980-0945-8 | PHP5 サンプル集 |
| 4-7981-0722-0 | XML 辞典     |
| 4-7981-0959-2 | PEAR 入門    |
| 4-8833-0000-1 | SQL リファレンス |
| 4-8833-0000-2 | SQL プチブック  |
| 4-8833-0000-3 | XML リファレンス |

9 rows in set (0.00 sec)

ROLLBACK 命令によって INSERT 命令が取り消されていることがお分かりになるはずです。

ただし、前述した TRUNCATE 命令や、後述する DROP DATABASE / TABLE 命令など、ロールバックできない（その場で変更が確定する）SQL 命令もありますので、注意してください。

また、トランザクションはロック管理とも密接に関連する機能で、データベース製品によって記法が異なる箇所があります。詳細は、データベース製品のドキュメントを参照してください。

## 3-8 ある問い合わせ結果に基づいて、削除したい DELETE 命令/サブクエリ

MySQL PostgreSQL SQLite SQL Server Oracle



SELECT、INSERT、UPDATE 命令同様、DELETE 命令でも削除対象のデータを特定するためのサブクエリを利用できます。

**Q** 貸し出し記録テーブル (rental) 上、一度も貸し出し記録のない書籍について、書籍情報テーブル (books) から該当データを削除してください。

**A** サブクエリを利用して、記述することが可能です。

```
> DELETE FROM
>   books
> WHERE
>   isbn
> NOT IN
> (
>   SELECT
>     isbn
>   FROM
>     rental
>   GROUP BY
>     isbn
> )
> ;
Query OK, 2 rows affected (0.03 sec)
```

| 書籍情報テーブル (books) |            |     |             | 貸し出し記録テーブル (rental) |         |               |          |
|------------------|------------|-----|-------------|---------------------|---------|---------------|----------|
| isbn             | title      | ... | category_id | id                  | user_id | isbn          | returned |
| 4-8833-0000-2    | SQL プチブック  | ... | S2222       | 9                   | B200503 | 4-8833-0000-2 | 1        |
| 4-8833-0000-3    | XML リファレンス | ... | X3333       | 21                  | B200504 | 4-7980-0945-8 | 0        |
| 4-0010-0000-6    | あひるの観察日記   | ... | A9999       | 23                  | A200405 | 4-8833-0000-3 | 0        |
| 4-0010-0000-7    | かえるの観察日記   | ... | H9999       | 13                  | A200404 | 4-8833-0000-2 | 1        |
| 4-7980-0945-8    | PHP5 サンプル集 | ... | P1111       | 11                  | A200401 | 4-8833-0000-2 | 1        |

抽出した結果に含まれない ISBN  
コードを持つレコードを削除

(次ページへ)

| isbn          |
|---------------|
| 4-8833-0000-2 |
| 4-8833-0000-3 |
| 4-7980-0945-8 |

isbn コードを  
グループ化  
して抽出



(前ページから)

| 書籍情報テーブル (books) |            |     |             |
|------------------|------------|-----|-------------|
| isbn             | title      | ... | category_id |
| 4-8833-0000-2    | SQL プチブック  | ... | S2222       |
| 4-8833-0000-3    | XML リファレンス | ... | X3333       |
| 4-7980-0945-8    | PHP5 サンプル集 | ... | P1111       |

## 構文を理解しよう

サブクエリを用いた **DELETE** 命令の基本的な構文は、以下の通りです。

### 構文 DELETE 命令 (サブクエリ)

**DELETE FROM** テーブル名 **WHERE** 列名 演算子 (サブクエリ);

更新しなさい

どこへ?

どのような条件で?

もう説明するまでもないかもしれませんが。サブクエリでは、具体的な条件値の代わりに **SELECT** 命令を記述します。サブクエリで抽出された結果に基づいて、条件値 (リスト) が決まってくるわけです。

なお、今回の例は P.166 でも取り上げた **EXISTS** 演算子による相関サブクエリとしても記述が可能です。相関サブクエリで書き換えた SQL 命令は、以下の通りです。

```
DELETE FROM
  books
WHERE
  NOT EXISTS
  (
    SELECT
      *
    FROM
      rental
    WHERE
      rental.isbn=books.isbn
  )
;
```

## SQL 命令を記述してみよう

サブクエリを利用した **DELETE** 命令を記述する流れは、以下の通りです。

### STEP ① 復習しよう (1)

まずは、サブクエリとなる部分の **SELECT** 命令を記述してみましょう。サブクエリの部分はもっとも初歩的な SQL 命令ですので、ここまで学習を進めてきた読者の皆さんならば、すぐさまに記述できると思います。

サブクエリでは、「貸し出し記録テーブル rental からすべての ISBN コードを重複がないように取り出し」ます。

この時点で、以下のような SQL 命令が記述できていれば正解です (ここでは **GROUP BY** 句を使用していますが、**DISTINCT** キーワードを使ってもかまいません)。これはまず横に置いておいて、続けて、以降のステップを見ていくことにしましょう。

```
SELECT
  isbn
FROM
  rental
GROUP BY
  isbn
```

### STEP ② 復習しよう (2)

続けて、サブクエリを受けて実行される本問い合わせを記述してみましょう。本問い合わせそのものも、これまでの復習にすぎませんので、これもこれまでの書き順にしたがって記述してみます。「ISBN コードが x、y いずれでもない書籍情報を削除して」みましょう。x、y はこの時点で仮に置いておく条件値です。



|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

この時点で、以下のような SQL 命令が記述できていれば正解です。

```
DELETE FROM
  books
WHERE
  isbn
NOT IN
(
  x,y
)
;
```

### STEP3 副問い合わせは？

最後は、例によって、STEP2 で仮置きしていた条件値「x,y」の部分に、STEP1 で作成しておいたサブクエリを埋め込むだけです。

```
DELETE FROM
  books
WHERE
  isbn
NOT IN
(
  SELECT
    isbn
  FROM
    rental
  GROUP BY
    isbn
)
;
```

完成!

これで1つの SQL 命令が完成です。

## マスタ・プラクティス



### 問1

所属部署テーブル (depart) がない所属部署に属する社員を社員テーブル (employee) から削除してください。以下の空欄を埋めて、SQL命令を完成させましょう。

```
①
employee
WHERE
  depart_id
②
(
  SELECT
    depart_id
  FROM
    ③
)
;
```

### 問2

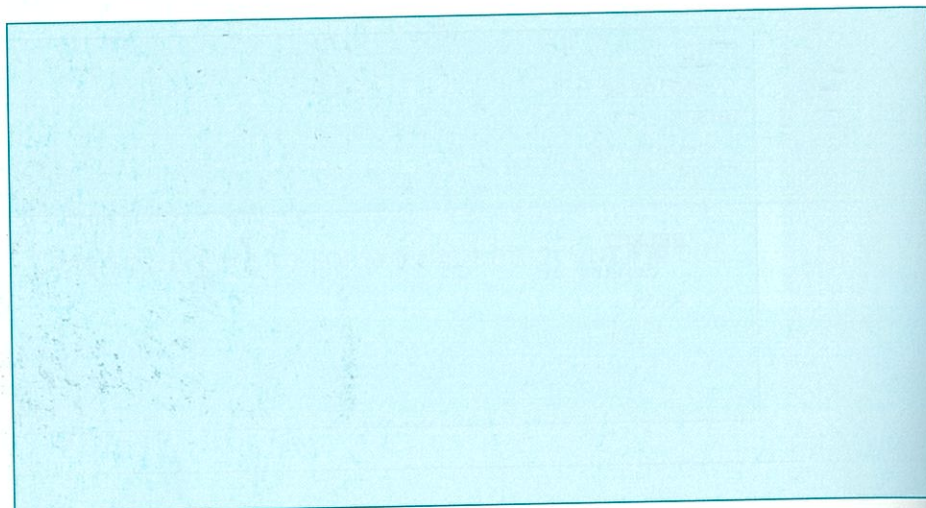
問1のSQL命令を相関サブクエリを使って書き換えてみましょう。

```
DELETE FROM
  employee
WHERE
  ①
(
  SELECT
    depart.depart_id
  FROM
    ②
  WHERE
    ③
)
;
```



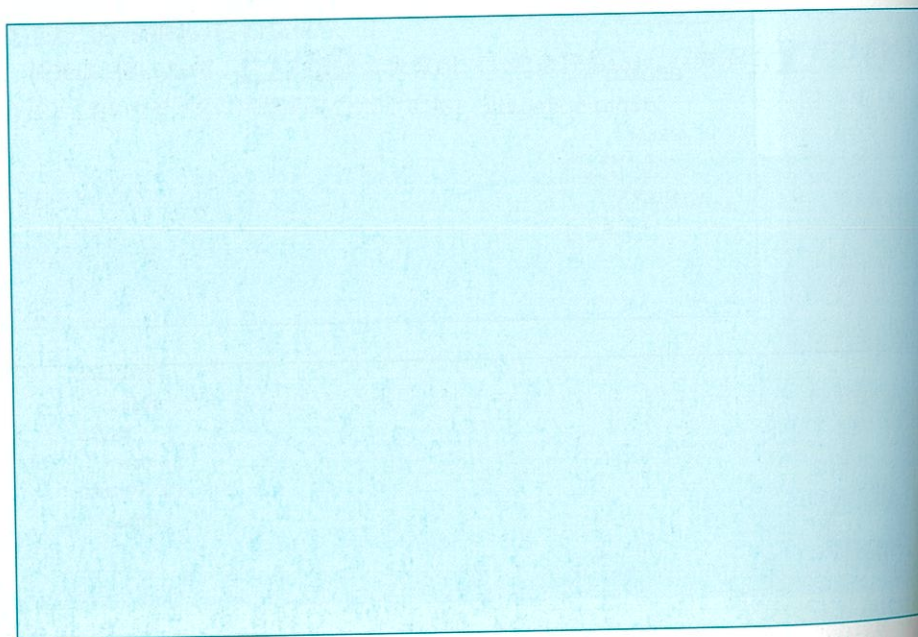
問 3

店舗テーブル (shop) に登録されていない店舗に関する情報を、月間売り上げテーブル (sales) から削除してください。



問 4

貸し出し記録テーブル (rental) 上、returned列が9 (紛失) であるレコードについて、対応する書籍情報テーブル (books) 上の書籍情報を削除してみましょう。



chapter

4

## データベース構造を 操作しよう

これまで SQL 命令の中でもデータそれ自体を操作するための「データ操作言語」について紹介してきました。アプリケーションを構築する中で、使用する 9 割がたはデータ操作命令ですが、データベースを理解するうえで、「データ定義言語」の理解は欠かせません。

データ定義言語については、データ型や細かな定義部分でデータベース製品ごとの差異が見られますが、本書ではごく基本的な部分と概念についてのみ学んでいくことにします。データベースごとの仕様については、各データベース製品のリファレンスやその他、専門のドキュメントを参照するようにしてください。

※ 注) なお、本章の例題や練習問題を実際に動作する場合には、新規のデータベースを作成して行うことをお勧めします。テーブルやインデックス定義が重複している場合、SQL 命令はエラーを返します。